

UNIVERSIDAD AUTÓNOMA DE QUERÉTARO

OPTIMIZACIÓN COMBINATORIA

Instructor: Dr. Arturo González Gutiérrez

Alumno: Ing. Daniel Eduardo González Alvarado

TAREA #5: Minimum Cost Spanning Tree

1. Introducción

En muchos problemas de optimización combinatoria es necesario encontrar la forma más eficiente de conectar un conjunto de nodos dentro de una red. Una herramienta fundamental para resolver este tipo de problemas es el *Árbol Generador de Costo Mínimo* (Minimum Cost Spanning Tree, MST). Este consiste en un subconjunto de aristas de un grafo conexo que conecta todos los vértices sin formar ciclos y cuyo costo total es mínimo. El estudio de los MST es importante en diversas aplicaciones como el diseño de redes de comunicación, redes eléctricas y sistemas de transporte. Para resolver este problema existen algoritmos eficientes basados en estrategias voraces, entre los más conocidos se encuentran los algoritmos de Kruskal, Prim y Sollin, los cuales construyen el árbol seleccionando progresivamente las aristas de menor costo sin generar ciclos en la estructura. Estos métodos permiten encontrar soluciones óptimas de manera eficiente incluso para grafos de gran tamaño.

- **Algoritmo de Kruskal:** Ordena todas las aristas del grafo de menor a mayor costo y las va agregando al árbol siempre que no formen un ciclo. El proceso continúa hasta que se han seleccionado $n - 1$ aristas, formando así el árbol.
- **Algoritmo de Prim:** Comienza con un vértice cualquiera y va construyendo el árbol agregando en cada paso la arista de menor costo que conecta un vértice del árbol con un vértice que aún no pertenece a él.
- **Algoritmo de Sollin:** Inicia con un bosque donde cada vértice es un componente independiente. En cada iteración, cada componente selecciona la arista de menor costo que lo conecta con otro componente, fusionando los componentes hasta formar un único árbol.

En esta ocasión, se abordará el problema de encontrar el árbol generador de costo mínimo para un grafo dado, utilizando los 3 algoritmos mencionados y comparando sus resultados.

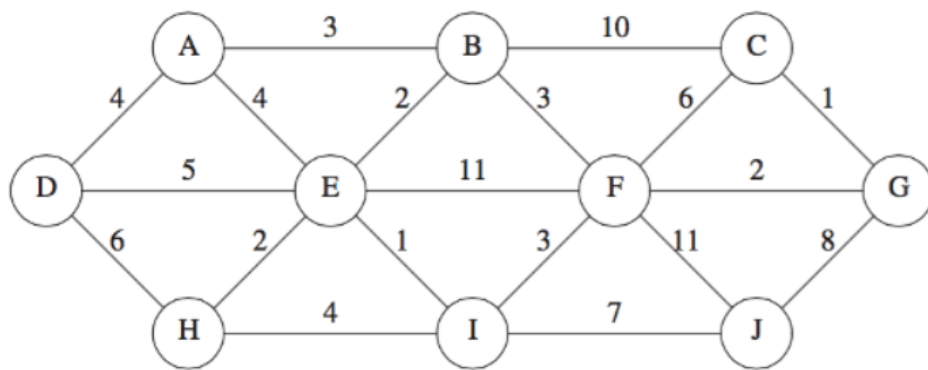


Figura 1: Grafo a resolver

2. Desarrollo

2.1. Algoritmo de Kruskal

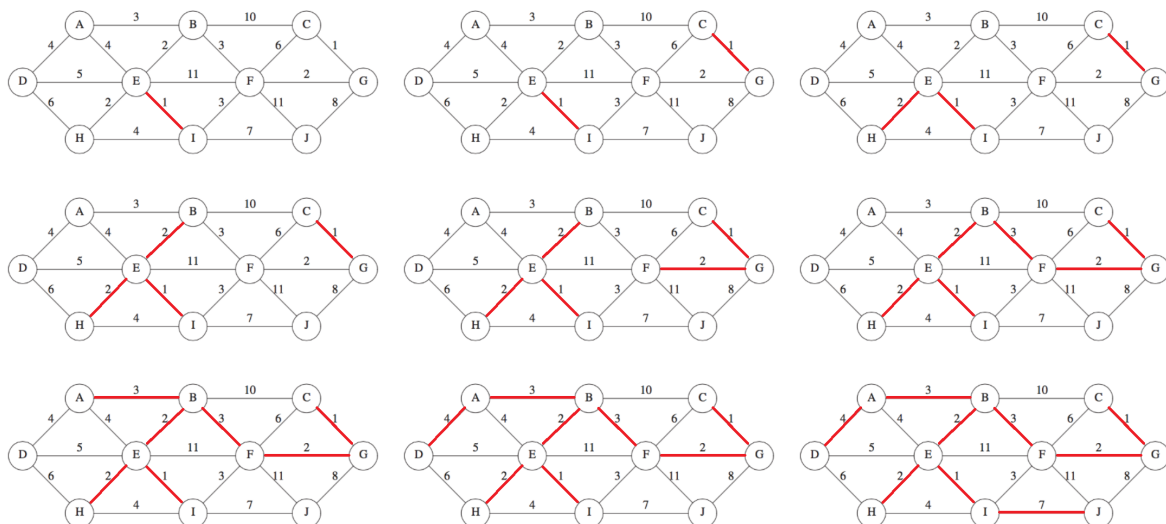


Figura 2: Procedimiento del algoritmo de Kruskal

Ruta seguida por el algoritmo de Kruskal:

Vértice	Grado
(E,I)	1
(C,G)	1
(E,H)	2
(E,B)	2
(F,G)	2
(B,F)	3
(F,I)	X
(B,A)	3
(D,A)	4
(A,E)	X
(H,I)	X
(D,E)	X
(D,H)	X
(F,C)	X
(I,J)	7
Total	25

Tabla 1: Costo total Algoritmo de Kruskal

2.2. Algoritmo de Prim

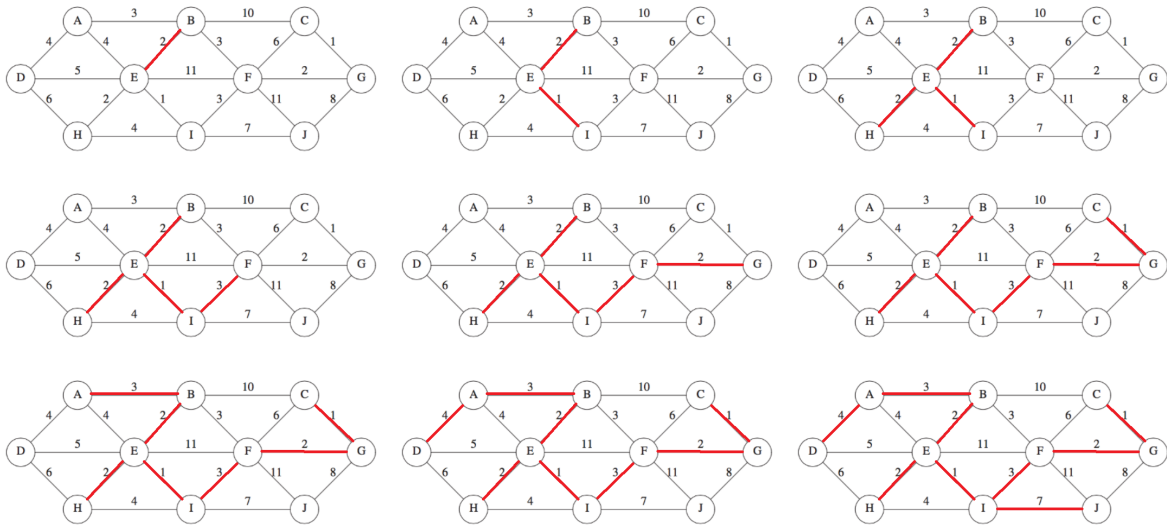


Figura 3: Procedimiento del algoritmo de Prim

Ruta seguida por el algoritmo de Prim:

Vértice	Grado
(B,E)	2
(E,I)	1
(E,H)	2
(I,F)	3
(F,G)	2
(G,C)	1
(B,A)	3
(A,D)	4
(I,J)	7
Total	25

Tabla 2: Costo total Algoritmo de Prim

2.3. Algoritmo de Sollin

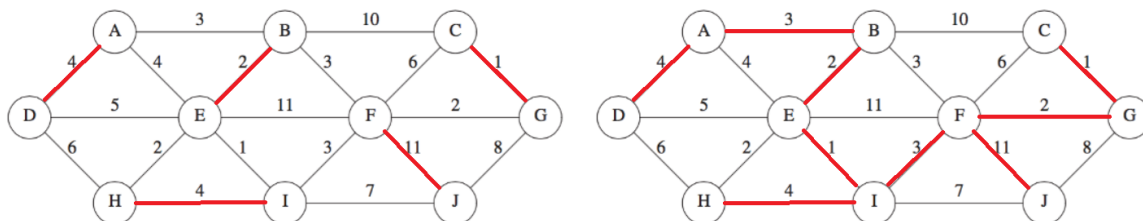


Figura 4: Procedimiento del algoritmo de Sollin

Ruta seguida por el algoritmo de Sollin:

Vértice	Grado	Paso
(B,E)	2	1
(C,G)	1	
(A,D)	4	
(H,I)	4	
(F,J)	11	
(A,B)	3	2
(E,I)	1	
(F,G)	2	
(E,F)	3	
Total	31	

Tabla 3: Costo total Algoritmo de Sollin

2.4. Resultados

2.4.1. Kruskal

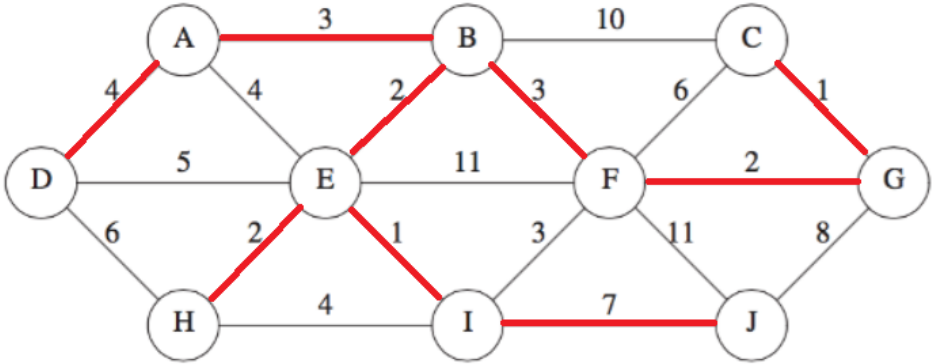


Figura 5: Arbol de Kruskal

2.4.2. Prim

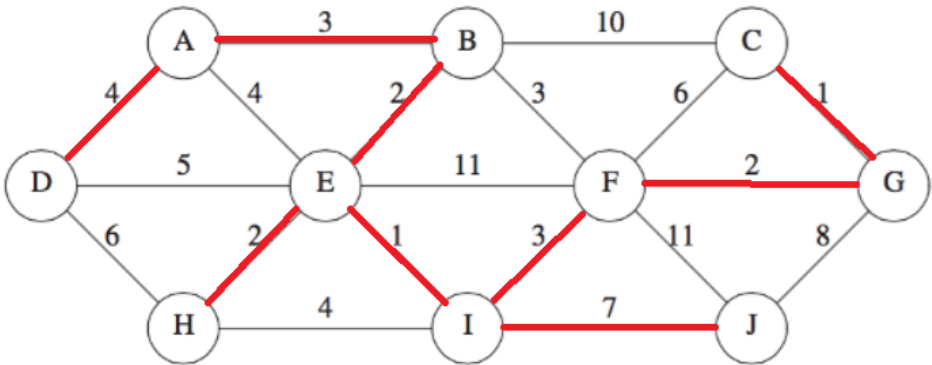


Figura 6: Arbol de Prim

2.4.3. Sollin

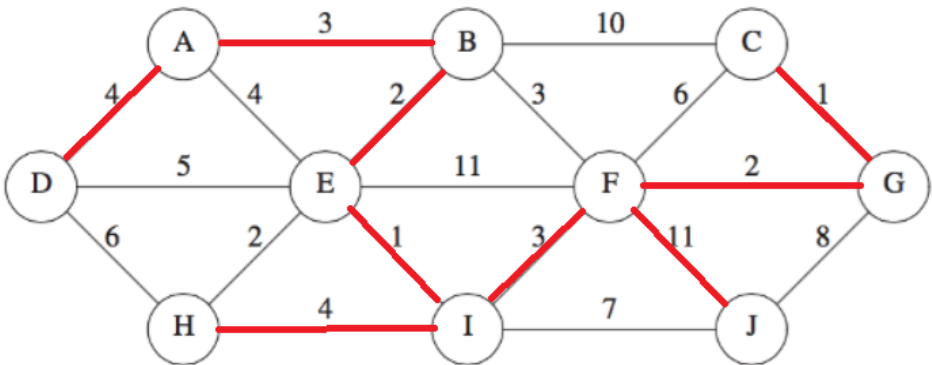


Figura 7: Arbol de Sollin

3. Conclusión

Como podemos ver los tres algoritmos, Kruskal, Prim y Sollin, lograron encontrar diferentes árboles generadores de costo mínimo para el mismo grafo, con costos totales de 25, 25 y 31 respectivamente.

Esto se debe a que cada algoritmo sigue una estrategia diferente para seleccionar las aristas, lo que puede llevar a resultados distintos dependiendo de la estructura del grafo y los costos de las aristas.